Enterprise Advanced RAG Learner Guide

Who This Guide Is For

This guide is for a new learner who has basic AI knowledge and wants to understand this project from start to finish.

It is not a short blog post.

It is a full learning guide that explains:

- what this project is
- why each part exists
- how the system works end to end
- what the main AI terms mean
- how the frontend controls map to backend behavior
- how the AWS deployment works
- how the tests, evals, CI, and load tests fit together

Use this guide together with:

- README.md
- WRITEUP.md
- AWS_DEPLOYMENT.md
- REBUILD_FROM_SCRATCH_GUIDE.md

---

1. What This Project Is

This project is an **enterprise-style AI assistant**.

It is not only a chatbot.

It combines:

- document-based question answering
- SQL question answering
- hybrid answers that combine SQL facts and document guidance
- advanced retrieval strategies
- safety checks
- admin monitoring
- cloud deployment

In simple words:

- if the answer is in documents, it uses **RAG**
- if the answer is in a database, it uses **SQL**
- if the answer needs both, it uses **hybrid**

This makes the project much more realistic than a basic “upload PDF and ask questions” demo.

---

2. What Problem It Solves

Real teams often store knowledge in different places:

- PDFs
- text files

- HTML docs
- runbooks
- policies
- structured tables in PostgreSQL

If a user asks:

- "What does the escalation policy say for P1 incidents?"

that is mostly a **document** question.

If a user asks:

- "How many P1 incidents were resolved last quarter?"

that is mostly a **database** question.

If a user asks:

- "How many P1 incidents were resolved last quarter and what does the escalation policy say for P1 incidents?"

that needs both:

- SQL for the count
- RAG for the policy text

This project was built to handle all three correctly.

---

3. What "RAG" Means

RAG stands for **Retrieval-Augmented Generation**.

Simple idea:

1. find relevant text from documents
2. send that text to the LLM
3. ask the LLM to answer using that retrieved text

Why do this?

Because the LLM should not answer only from memory. It should answer from the actual project documents when possible.

Example:

Question:

```
What is the difference between a ConfigMap and a Secret in Kubernetes?
```

RAG flow:

1. retrieve Kubernetes chunks about ConfigMap and Secret
2. pass those chunks to the model
3. generate a grounded answer with sources

---

4. Main AI Terms In This Project

4.1 Dense Retrieval

Dense retrieval uses embeddings.

Embeddings are numerical vectors created from text.

Dense retrieval is good for:

- semantic meaning

- paraphrased questions
- conceptual questions

Example:

- “How do I make sure my app keeps running even if a server dies?”

This question may not match exact keywords like “high availability” or “replicas,” but dense retrieval can still find related ideas.

4.2 Sparse Retrieval

Sparse retrieval is more keyword-focused.

It is good for:

- exact terms
- operational error strings
- command names
- short technical phrases

Example:

- `ErrImagePull`
- `OOMKilled`
- `CrashLoopBackOff`

These are exact technical signals, so sparse retrieval helps a lot.

4.3 Hybrid Retrieval

Hybrid retrieval combines:

- dense retrieval
- sparse retrieval

Why use it?

Because enterprise questions often need both:

- exact lexical matching
- semantic similarity

In this project, **hybrid + rerank** became the strongest general-purpose default.

4.4 Reranking

Reranking means:

1. retrieve candidate chunks
2. score them again more carefully
3. move the best chunks to the top

Why use it?

Initial retrieval may find many chunks, but some are noisy or only partially relevant.

Reranking improves the final chunk order.

4.5 HyDE

HyDE stands for **Hypothetical Document Embeddings**.

Simple idea:

1. the LLM imagines a likely answer
2. that synthetic answer is embedded
3. retrieval uses that representation to find better source chunks

Why use it?

Sometimes the user asks a vague or conceptual question.

Example:

- "How do I make sure my app keeps running even if a server dies?"

This may retrieve better if the system first imagines a likely answer about:

- replicas
- availability
- failover
- Kubernetes scheduling

4.6 CRAG

CRAG here means a **corrective retrieval check**.

Simple idea:

1. retrieve chunks
2. evaluate whether retrieval quality looks strong enough
3. if retrieval looks weak, treat the answer more cautiously

Why use it?

Even if the model can answer, the retrieved evidence might be weak.

CRAG gives the system a chance to detect:

- low-quality retrieval
- partially relevant retrieval
- uncertain grounding

In the metadata, you may see:

- `crag_score`
- `crag_label`

Example labels:

- `highly_relevant`
- `somewhat_relevant`

4.7 SRAG / Self-Reflective RAG

SRAG means the system can reflect on whether the answer or question needs improvement.

Simple idea:

1. answer once
2. judge whether the result is weak or uncertain
3. refine the question or reasoning if needed

Why use it?

Some troubleshooting questions are ambiguous.

Example:

- "My Kubernetes app keeps restarting, and I am not sure whether the root cause is the liveness probe, OOMKilled, or a bad image. How should I troubleshoot it step by step?"

This is a good SRAG-style question because:

- the user is uncertain
- several causes are possible
- a step-by-step reasoning process helps

4.8 Intent Routing

Intent routing means deciding what kind of question this is.

In this project, the main routes are:

- `rag`
- `sql`
- `hybrid`

The router uses heuristics in app/services/router_service.py.

Examples:

- "How many users are there?" -> likely `sql`
- "What does our escalation policy say?" -> likely `rag`
- "How many incidents were resolved and what does the policy say?" -> likely `hybrid`

4.9 Text-to-SQL

Text-to-SQL means converting a natural language question into SQL.

Example:

Question:

- "How many P1 incidents were resolved last quarter?"

Possible SQL idea:

```
select count(*) from incidents where severity = 'P1' and status = 'resolved' ...
```

4.10 Human-In-The-Loop SQL Approval

This project does **not** automatically trust all generated SQL.

Instead:

1. generate SQL
2. pause
3. show SQL to the user
4. run only after approval

This is safer and more realistic for enterprise systems.

4.11 `sqlglot` SQL Validation

`sqlglot` is used to parse SQL as a structured syntax tree.

Why is this better than regex only?

Because regex checks can miss dangerous patterns.

This project uses `sqlglot` to allow only safe read-only query shapes.

Blocked examples include:

- `DROP`
- multi-statement SQL
- dangerous write paths

---

5. High-Level Architecture

At a high level, the system looks like this:

```
User -> Frontend -> FastAPI -> Guardrails -> Router -> LangGraph
                                                |-> RAG path
                                                |-> SQL path
                                                |-> Hybrid path
```

Main systems:

- **Streamlit** frontend
- **FastAPI** backend
- **LangGraph** orchestration
- **PostgreSQL** for structured data and auth
- **Qdrant** for vector search
- **OpenAI** for embeddings and generation
- **S3** for document storage
- **CloudWatch** for production monitoring

---

6. Why These Technologies Were Chosen

6.1 FastAPI

Chosen because:

- simple and clean Python API framework
- good request/response validation
- easy to test

6.2 Streamlit

Chosen because:

- fast way to build an AI control panel
- easy to show query controls, tabs, sources, metadata, and admin features

6.3 PostgreSQL

Chosen because:

- strong relational database
- realistic for enterprise structured data
- needed for SQL questions

6.4 Qdrant

Chosen because:

- purpose-built vector database
- stores embeddings for semantic retrieval
- supports enterprise-style RAG architecture

6.5 LangGraph

Chosen because:

- routing and workflow are not linear
- SQL flow can pause and resume
- hybrid flow combines multiple branches

6.6 AWS ECS Fargate

Chosen because:

- production deployment without managing Kubernetes for the app itself
- simpler than operating a full cluster
- still credible and realistic for deployment

Important note:

The **content** of the assistant is Kubernetes-focused, but the **hosting** platform for this app is AWS ECS Fargate.

---

7. Basic Setup And Why Each Step Exists

7.1 Create a Virtual Environment

```
python -m venv .venv
.\.venv\Scripts\activate
```

Why?

- keeps project packages isolated
- avoids dependency conflicts with other Python projects

7.2 Install Dependencies

```
pip install -e ".[dev]"
```

Why?

- installs the app in editable mode
- includes dev tools like `pytest`
- lets code changes work immediately without reinstalling

7.3 Configure Environment Variables

Copy `.env.example` to `.env`.

Why?

The app needs configuration for:

- OpenAI API access
- PostgreSQL
- Qdrant
- JWT auth
- optional Redis cache
- optional web fallback

Without this, the app cannot connect to its services.

7.4 Start Local Infra

```
docker compose up -d
```

Why?

Local development needs services such as:

- PostgreSQL
- Qdrant

The application code depends on them.

7.5 Run the Backend

```
python -m uvicorn app.main:app --reload --port 8003
```

Why?

This starts the FastAPI application locally.

Useful endpoints:

- `/docs`
- `/health`

7.6 Run the Frontend

The Streamlit app is in scripts/streamlit_app.py.

Why?

This is the interactive UI for:

- login/register
- querying
- uploading docs
- viewing sources/chunks/raw JSON
- admin checks

---

8. Project Structure And What Each Part Does

8.1 `app/api/`

This contains API routes.

Important routes:

- auth
- documents
- query
- admin

8.2 `app/core/`

This contains:

- shared graph state
- LangGraph orchestration

8.3 `app/services/`

This is the main business logic layer.

Examples:

- ingestion
- retrieval
- SQL
- caching
- auto retrieval
- conversation context

8.4 `app/security/`

This contains safety and guardrail logic.

Examples:

- input checks
- moderation/redaction
- token budgeting
- output validation

8.5 `app/storage/`

This handles where files are stored.

Examples:

- local storage
- S3 storage

8.6 `tests/`

This contains automated tests.

8.7 `Eval/`

This contains evaluation tooling for retrieval and answer quality workflows.

8.8 `deploy/`

This contains deployment-related JSON and dashboard definitions.

8.9 `loadtests/`

This contains k6 scripts and results for performance validation.

---

9. Ingestion Flow: How Documents Enter the System

The ingestion flow is handled by app/services/ingestion_service.py.

Simple flow:

1. user uploads a file
2. compute a content hash
3. check whether this content was already indexed
4. save original file to storage
5. parse document text
6. chunk the text
7. create embeddings
8. upsert vectors into Qdrant
9. store metadata

Why compute a content hash?

Because two files may have different names but identical content.

This avoids indexing duplicates.

Why store original files too?

Because retrieval is based on chunks, but the original file is still valuable for:

- traceability
- auditing
- future reprocessing

---

10. Why Chunking Is Needed

Documents are too large to send whole into the model every time.

So the project splits them into smaller pieces called **chunks**.

Why chunk?

- smaller retrieval units
- better matching
- lower token cost
- more focused context for answers

Example:

A long Kubernetes guide might contain sections on:

- ConfigMap
- Secret
- Pod lifecycle
- Service discovery

Only the relevant chunks should be retrieved for a ConfigMap question.

---

11. Query Flow: What Happens When a User Asks a Question

When a user sends a question to `/query`, the system does not immediately call the LLM.

It first performs several steps:

1. auth check
2. rate limit check
3. token budget check
4. input restructuring if needed
5. input safety check
6. moderation/redaction
7. follow-up rewrite if conversation context exists
8. intent routing
9. retrieval strategy selection
10. graph execution
11. output validation
12. response assembly

This is important.

A production AI system should not be only:

- user question -> LLM -> answer

It needs control layers around the model.

---

12. Intent Routing: RAG vs SQL vs Hybrid

The intent router lives in app/services/router_service.py.

It uses heuristic patterns.

It looks for signals such as:

- SQL words like `how many`, `count`, `average`
- SQL entities like `incidents`, `users`, `cluster`, `SLA`
- RAG clues like `policy`, `runbook`, `guide`, `Kubernetes`

Decision examples:

- “How many users are there?” -> `sql`
- “What does our escalation policy say?” -> `rag`
- “How many incidents were resolved and what does the policy say?” -> `hybrid`

Why is routing important?

Because document retrieval and SQL generation solve different problems.

If routing is wrong, the answer will be weaker even if the model is strong.

---

13. Auto Mode: How Backend-Owned Retrieval Selection Works

The auto retrieval logic lives in app/services/auto_retrieval.py.

This file is one of the most important parts of the project.

It decides:

- which search mode to use
- whether rerank should be on
- whether HyDE should be on
- whether CRAG should be on
- whether SRAG should be on

13.1 Why Auto Mode Exists

Manual mode is useful for demos.

But in production, users should not need to know retrieval theory.

So auto mode tries to choose the best strategy from the question itself.

13.2 What Signals Auto Mode Uses

The code checks for signals such as:

- troubleshooting terms
- guidance terms
- conceptual wording
- exact technical tokens
- action-oriented wording
- uncertainty and reflection clues
- vagueness

13.3 Main Auto Rules

Short exact-token query

Example:

- `ErrImagePull`

Likely choice:

- sparse
- rerank on
- CRAG on

Why?

Exact operational term. Keyword matching matters most.

Direct object comparison

Example:

- “What is the difference between a ConfigMap and a Secret in Kubernetes?”

Likely choice:

- hybrid

- rerank on
- HyDE off

Why?

This is a direct comparison with exact domain objects. Hybrid gives both lexical and semantic support.

Strong conceptual question

Example:

- “What is the best way to think about service discovery in Kubernetes?”

Likely choice:

- dense
- rerank on
- HyDE on

Why?

Conceptual phrasing benefits from semantic retrieval.

Ambiguous troubleshooting question

Example:

- “My Kubernetes app keeps restarting, and I am not sure whether the root cause is the liveness probe, OOMKilled, or a bad image. How should I troubleshoot it step by step?”

Likely choice:

- hybrid
- rerank on
- CRAG on
- SRAG on

Why?

- multiple possible causes
- uncertainty is explicit
- step-by-step troubleshooting needs stronger reasoning support

Very vague query

Example:

- “Why is this wrong?”

Likely choice:

- dense
- HyDE on
- CRAG on
- SRAG on

Why?

The question is too vague and benefits from semantic expansion plus reflection.

13.4 Important Auto Example: Why “how do i scale” Did Not Turn On SRAG

This came up during debugging.

Question:

- `how do i scale`

Why SRAG was not activated:

- it is a short operational action question
- it does not express uncertainty strongly enough
- it does not list multiple possible causes

So auto mode treated it as:

- hybrid
- rerank on
- CRAG on
- SRAG off

That behavior is correct under the current logic.

13.5 Auto Mode Metadata

When auto mode runs, the response can expose:

- `requested_search_mode`
- `resolved_search_mode`
- `resolved_enable_rerank`
- `resolved_enable_hyde`
- `resolved_enable_crag`
- `resolved_enable_self_reflective`
- `auto_enabled`
- `auto_strategy_reason`
- `auto_applied_flags`

This is very useful for debugging and learning.

---

14. Manual Mode: What Each Frontend Control Means

The frontend controls are defined in scripts/streamlit_app.py.

In Manual mode, the user can control:

- Search Mode
- Top K
- Enable Rerank
- Enable HyDE
- Enable CRAG
- Enable SRAG

14.1 Search Mode

Choices:

- `dense`
- `sparse`
- `hybrid`

Expected use:

- `dense`: semantic/conceptual questions
- `sparse`: exact keywords and troubleshooting tokens

- `hybrid`: safest general default

14.2 Top K

This is the number of chunks retrieved.

Why it matters:

- too low: might miss useful context
- too high: might add noise

In this project, many examples use `Top K = 5`.

14.3 Enable Rerank

When on:

- retrieved chunks are rescored and reordered

Expected effect:

- better final chunk quality
- especially helpful on noisy corpora

14.4 Enable HyDE

When on:

- retrieval uses hypothetical-answer expansion

Expected effect:

- better results for paraphrased or conceptual questions

14.5 Enable CRAG

When on:

- retrieval quality is evaluated
- metadata shows CRAG-related signals

Expected effect:

- better awareness when retrieval is weak or uncertain

14.6 Enable SRAG

When on:

- system can reflect on uncertainty and improve reasoning

Expected effect:

- more useful on ambiguous troubleshooting questions
- usually slower than simpler paths

---

15. Preset Queries And Why They Exist

The frontend has preset use cases.

These are not random examples. They are there to demonstrate different system behaviors.

15.1 Hybrid Default

Question:

- “How many P1 incidents were resolved last quarter and what does our escalation policy say for P1 incidents?”

Purpose:

- show hybrid SQL + RAG flow

Expected idea:

- SQL for count
- RAG for policy

15.2 RAG Policy

Question:

- “What does our escalation policy say for P1 incidents?”

Purpose:

- show document-grounded lookup

15.3 SQL Count

Question:

- “How many P1 incidents were resolved last quarter?”

Purpose:

- show SQL route and possibly approval flow

15.4 Troubleshooting

Question:

- “What does ErrImagePull mean and how do I fix it?”

Purpose:

- show operational troubleshooting with exact technical tokens

15.5 HyDE Search

Question:

- “How do I make sure my app keeps running even if a server dies?”

Purpose:

- show semantic retrieval expansion

15.6 SRAG Reflection

Question:

- “My Kubernetes app keeps restarting, and I am not sure whether the root cause is the liveness probe, OOMKilled, or a bad image. How should I troubleshoot it step by step?”

Purpose:

- show ambiguous troubleshooting
- show why SRAG is valuable

---

16. What “Active Config” Means in the UI

The UI shows an **Active Config** panel.

This is very important.

It tells the user what the system actually used.

Possible fields:

- Experience Mode
- Search Mode
- Top K
- Rerank
- CRAG
- HyDE
- SRAG

In Auto mode, these values may come from backend-resolved metadata.

In Manual mode, these values mostly reflect the buttons selected by the user.

Why is this useful?

Because it lets a learner compare:

- what they asked
- what the backend decided
- what retrieval strategy was actually used

---

17. What the Response Tabs Mean

The frontend shows several tabs for a response.

17.1 Answer Tab

This shows:

- final answer
- route
- confidence
- cache status
- cost-saved values

17.2 Sources Tab

This shows the sources used by the answer.

Why it matters:

- helps you trust the answer
- helps you debug wrong answers

17.3 Chunks Tab

This shows chunk previews.

You may also see:

- score
- retrieval score
- rerank score

Why it matters:

- lets you inspect whether retrieval found the right evidence

17.4 Metadata Tab

This shows summary metadata such as:

- route
- cache hit
- reflection iterations
- CRAG score
- web fallback usage

17.5 Raw JSON Tab

This shows the full backend response.

Why it matters:

- excellent for debugging
- helpful for learning how the API is structured

---

18. What Confidence Means

The response shows a confidence value.

This is a system-level estimate, not a perfect truth score.

It is useful as a hint, but should not be blindly trusted.

A strong workflow is:

1. read the answer
2. inspect the sources
3. inspect the chunks if the answer looks suspicious

---

19. What Cost-Saved Means

The response may show:

- `cost_saved`
- `llm_cost_saved`
- `infra_cost_saved`

These are useful when the answer came from cache.

Meaning:

- the system avoided recomputing some work
- that saved model or infra cost

If the response is fresh, these values should be zero.

That is correct behavior.

---

20. Conversation Context and Follow-Up Questions

This project supports follow-up questions.

Example:

First question:

- “What is the difference between a ConfigMap and a Secret in Kubernetes?”

Follow-up:

- “For the non-sensitive one, if I inject it as environment variables instead of a volume, will updates show up automatically?”

The system tries to rewrite the follow-up into a clearer question before continuing.

This logic is handled by a conversation-context service and is tested.

Why is this useful?

Because users often do not repeat full nouns in real conversations.

They say things like:

- “that same object”
- “the non-sensitive one”
- “would you still use it for a database password?”

Without context rewriting, many follow-up questions would be weaker.

---

21. LangGraph Wiring: Why the Workflow Is Not Linear

The orchestration is in app/core/graph.py.

Why use a graph?

Because the workflow can branch.

Routes:

- `rag`
- `sql`
- `hybrid`

Key nodes:

- route intent
- generate SQL
- request SQL approval
- run SQL
- run RAG
- retrieve RAG for hybrid
- run hybrid

Why is the graph useful?

Because SQL flow can pause for approval and then resume.

That is harder to model cleanly in a simple chain.

22. SQL Path Explained Step by Step

For a SQL question:

1. router decides `sql`
2. system generates SQL
3. SQL is validated
4. system may pause with a `pending_sql` block
5. user approves
6. SQL executes
7. natural-language answer is returned

The graph code also formats SQL answers into readable text.

Example:

Instead of returning:

```
[{"count": 50}]
```

it can return something like:

- “50 P1 incidents were resolved last quarter.”

This improves user experience.

23. Hybrid Path Explained Step by Step

For a hybrid question:

1. router decides `hybrid`
2. run RAG retrieval for document context
3. generate SQL for structured facts
4. approval step if needed
5. execute SQL
6. combine RAG context and SQL results
7. generate final grounded answer

This is one of the strongest parts of the project because it shows the system can combine:

- narrative guidance from documents
- numeric facts from tables

24. Guardrails and Safety Layers

This project has many safety layers.

They are not just extras. They are part of making the system production-minded.

Examples:

- request validation
- JWT authentication
- rate limiting
- token budgets
- input safety rules

- moderation/redaction
- output validation

Why so many layers?

Because AI systems can fail in many ways:

- too many requests
- unsafe instructions
- prompt injection attempts
- information leakage
- unsafe generated outputs

---

25. Caching: Why It Exists and What Is Cached

This project uses multiple cache types.

Examples:

- intent cache
- RAG answer cache
- SQL generation cache
- SQL result cache
- embedding cache

Why cache?

- faster repeated responses
- lower model cost
- lower infra cost

Important design lesson:

Document deduplication and query cache are not the same thing.

- document dedupe avoids re-indexing the same file
- query cache avoids recomputing the same answer path

---

26. Production Deployment: What Was Chosen and Why

The production deployment uses AWS.

Main pieces:

- ECS Fargate
- ECR
- ALB
- Route 53
- ACM
- RDS PostgreSQL
- Qdrant Cloud
- S3
- Secrets Manager
- CloudWatch

26.1 Why ECS Fargate

Chosen because:

- no need to manage EC2 hosts
- simpler than running our own cluster
- still gives production credibility

26.2 Why ECR

ECR stores Docker images for deployment.

26.3 Why ALB

ALB handles:

- HTTP/HTTPS traffic
- routing to healthy targets

26.4 Why Route 53 and ACM

These provide:

- DNS
- HTTPS certificates

That is why the deployed URLs look professional:

- `https://api.aiscientificlabs.com`
- `https://rag.aiscientificlabs.com`

26.5 Why RDS

RDS is used for PostgreSQL so the database is managed in AWS.

26.6 Why Qdrant Cloud

Qdrant Cloud avoids self-hosting the vector database for this project.

That reduces operational complexity.

26.7 Why CloudWatch

CloudWatch gives:

- logs
- alarms
- metrics
- dashboard screenshots during load tests

This is very important for proving the deployed system is healthy.

---

27. Production Deployment Stages

The deployment guide breaks the work into stages.

High-level sequence:

1. configure AWS CLI
2. create ECR repositories
3. build images
4. push images
5. create RDS
6. create S3 bucket
7. prepare Qdrant Cloud
8. create Secrets Manager secret
9. create ECS cluster
10. create IAM roles
11. create CloudWatch log groups
12. register task definitions
13. create target groups
14. create ALBs
15. create ECS services
16. add Route 53 and ACM
17. validate backend and frontend

Why do this in order?

Because cloud deployments often fail if lower-level resources are missing.

Examples:

- missing log groups
- missing secret permissions
- target group health mismatches
- wrong subnet alignment

---

28. CloudWatch Dashboard: What It Shows

The project created a dashboard called:

- `new-adv-rag-ops`

It includes:

- manual alarm status
- backend ALB response time
- backend ECS CPU and memory
- RDS CPU and connections
- backend ALB 5xx and request count
- backend target group health
- streamlit ECS CPU
- backend request logs

Why is this useful?

Because during load tests you can see:

- whether traffic is reaching the system
- whether latency is increasing
- whether CPU is rising

- whether targets remain healthy

28.1 The Seven Manual Alarms

The "manual alarm status" widget tracks seven alarms created specifically for this project, not default AWS alarms:

- `backend-alb-5xx`
- `backend-alb-response-time-high`
- `backend-ecs-cpu-high`
- `backend-ecs-memory-high`
- `rds-connections-high`
- `rds-cpu-high`
- `streamlit-ecs-cpu-high`

28.2 Manual Alarms vs. AWS Autoscaling Alarms

There is a subtlety worth knowing here.

ECS service autoscaling automatically creates its own alarms, named like `TargetTracking-service/...`.

These are:

- AWS-managed
- created automatically by the autoscaling policy
- not the same as the seven manual alarms listed above

Important lesson:

Do not mistake the appearance of `TargetTracking-service/...` alarms in the alarm list for deployment drift or misconfiguration. They are expected. Confusing "an alarm I didn't create" with "an alarm that shouldn't be there" is an easy mistake to make when first looking at a live AWS account, and it is worth being able to explain the difference clearly.

---

29. Load Testing: What Was Run and What It Means

The final summary is in loadtests/results/FINAL_LOADTEST_RESULTS_20260708.md.

Main scripts:

- `health.js`
- `query_rag.js`
- `mixed_smoke.js`

29.1 Health Test

Purpose:

- test basic availability

Result:

- requests: `58`
- error rate: `0.00%`
- average latency: `62.62ms`
- p95 latency: `69.11ms`
- threshold: pass

Meaning:

- the backend health endpoint was stable and fast

29.2 RAG Test

Purpose:

- test authenticated RAG query path

First run:

- p95 was too high
- threshold failed

Second run:

- average latency: `4.78s`
- p95 latency: `5.14s`
- threshold passed

Meaning:

- RAG worked functionally
- latency improved on repeated run
- the live RAG path is usable but still not extremely fast

29.3 Mixed Smoke Test

Purpose:

- test mixed RAG + SQL approval + hybrid path

Result:

- error rate: `0.00%`
- average latency: `10.88s`
- p95 latency: `34.05s`
- threshold failed

Meaning:

- the full end-to-end system worked correctly
- no HTTP failures occurred
- latency is the main remaining optimization area

This is an important learning point:

correctness and availability can be good even when latency still needs work.

---

30. What the Load Test Results Prove

The load-test session proved:

- health endpoint is stable
- RAG path works
- SQL approval flow works
- hybrid flow works
- no HTTP failures were seen in validated runs
- the system stayed healthy in CloudWatch and ECS during tests

It also showed:

- mixed end-to-end latency is currently the main improvement area

---

31. GitHub Actions and CI

The repo uses GitHub Actions.

Main CI goals:

- run tests automatically
- catch regressions before changes are trusted
- keep the project professional and repeatable

The CI workflow runs:

- `pytest -q`
- eval-smoke jobs

Important lesson from this project:

At one point, CI failed because some tests accidentally depended on a local Postgres-backed pending SQL store.

Why that mattered:

- tests passed locally
- failed in GitHub Actions

The fix was to make the tests independent from local-only infrastructure.

This is a very real production engineering lesson.

---

32. Evals: What They Are and Why They Matter

This project includes evaluation workflows under `Eval/`.

Why evals matter:

- tests check code correctness
- evals check AI behavior quality

Examples of eval focus:

- retrieval quality
- answer relevance
- grounding behavior

Recorded smoke results from validation work:

- baseline profile: faith about `0.598`
- hybrid profile: faith about `0.600`
- hybrid SQL profile: faith about `0.614`
- CRAG profile: faith about `0.585`

How to interpret that:

- these are not perfect “scores of truth”
- they are comparison tools
- they help detect regressions and compare strategies

Key practical conclusion from the project:

- `hybrid + rerank` is the strongest general default
- HyDE, CRAG, and SRAG are best used conditionally, not always-on

---

33. What the Tests Cover

Below is a learner-friendly summary of the test suite.

33.1 Auth Tests

Examples:

- valid login shape
- auth enforcement

Purpose:

- ensure protected routes are not open accidentally

33.2 Auto Retrieval Tests

File:

- `tests/test_auto_retrieval.py`

Covers:

- sparse choice for short exact-token queries
- hybrid for policy questions
- dense + HyDE for semantic questions
- hybrid for direct object comparisons
- dense + HyDE + CRAG + SRAG for vague questions
- SRAG activation for ambiguous troubleshooting questions

Purpose:

- prove auto mode is making the intended strategy decisions

33.3 Conversation Context Tests

File:

- `tests/test_conversation_context.py`

Covers:

- follow-up rewriting for ConfigMap
- follow-up rewriting for Secret
- expansion of earlier comparison pairs

Purpose:

- preserve conversational context quality

33.4 Ingestion Tests

File:

- `tests/test_ingestion_service.py`

Covers:

- duplicate file ingestion is skipped

Purpose:

- prevent duplicate indexing

33.5 Hybrid Routing Tests

File:

- `tests/test_hybrid_routing.py`

Covers:

- correct hybrid intent classification
- hybrid combination logic
- behavior when SQL is not approved

33.6 Query API Tests

File:

- `tests/test_query_api.py`

Covers:

- auth requirement
- unsafe input rejection
- auto strategy resolved before graph execution
- follow-up rewrite applied before graph
- approved cache reuse
- SQL pending approval response
- SQL execution resume path
- explicit SQL execution errors
- explicit missing approval-context errors
- hybrid SQL pending behavior

Purpose:

- verify the main API contract and tricky flow edges

33.7 RAG Source Filtering Tests

File:

- `tests/test_rag_source_filtering.py`

Covers:

- noisy unrelated chunks are dropped
- negative rerank tails are dropped
- better named sources are preferred
- non-Kubernetes noise is filtered out when canonical Kubernetes sources exist

Purpose:

- improve answer grounding and reduce noisy chunks

33.8 SQL Service Tests

File:

- `tests/test_sql_service.py`

Covers:

- safe SELECT patterns allowed
- dangerous SQL blocked

- question cleanup for SQL generation
- heuristic SQL behavior
- generation cost reporting

Purpose:

- verify SQL safety and route quality

33.9 Vector Store and Reranking Tests

Files:

- `tests/test_vector_store.py`
- `tests/test_reranking.py`

Covers:

- hybrid boosting for troubleshooting matches
- lexical rescue for exact operational terms
- better runbook promotion
- sparse query expansion for terms like `ErrImagePull` and `OOMKilled`

Purpose:

- improve retrieval quality for realistic operations questions

---

34. Example Questions to Learn Each Feature

34.1 Dense / Semantic

- “How do I make sure my app keeps running even if a server dies?”

Expected idea:

- dense or dense-like behavior
- HyDE helpful

34.2 Sparse / Exact Token

- “What does ErrImagePull mean?”

Expected idea:

- exact token should matter
- sparse or hybrid with lexical support

34.3 Hybrid Comparison

- “What is the difference between a ConfigMap and a Secret in Kubernetes?”

Expected idea:

- hybrid
- rerank on

34.4 SRAG-Style Question

- “My Kubernetes app keeps restarting, and I am not sure whether the root cause is the liveness probe, OOMKilled, or a bad image. How should I troubleshoot it step by step?”

Expected idea:

- hybrid
- CRAG on
- SRAG on

34.5 SQL Question

- “How many P1 incidents were resolved last quarter?”

Expected idea:

- SQL route
- approval flow may appear

34.6 Hybrid SQL + RAG

- “How many P1 incidents were resolved last quarter and what does our escalation policy say for P1 incidents?”

Expected idea:

- hybrid route
- SQL + document context combined

---

35. How to Read the System Like a New Learner

If you are new, study in this order:

1. read `README.md`
2. read this learner guide
3. inspect `scripts/streamlit_app.py`
4. inspect `app/services/router_service.py`
5. inspect `app/services/auto_retrieval.py`
6. inspect `app/services/ingestion_service.py`
7. inspect `app/core/graph.py`
8. inspect `app/services/rag_service.py`
9. inspect `app/services/sql_service.py`
10. inspect the tests
11. read `AWS_DEPLOYMENT.md`
12. read the load-test result summary

This order helps because it moves from:

- user experience
- to routing logic
- to backend implementation
- to production operations

---

36. Key Design Decisions and Why They Matter

36.1 Why Not Just Use One Retrieval Mode

Because different question types need different strengths.

Examples:

- exact tokens need lexical power
- conceptual questions need semantic power
- enterprise corpora often need both

36.2 Why Not Always Turn On Every Feature

Because more features do not always mean better quality.

Always-on advanced retrieval can increase:

- latency
- noise
- cost
- complexity

That is why this project uses:

- manual mode for demos
- auto mode for backend policy

36.3 Why Human SQL Approval Was Added

Because SQL is riskier than simple document retrieval.

Even read-only SQL should be more controlled in a serious system.

36.4 Why CloudWatch and Load Tests Matter

Because a local demo is not enough.

Production-minded work means:

- deploy it
- observe it
- test it under traffic
- save evidence

---

37. What the Current Results Say About the System

Overall, the system looks strong in:

- architecture depth
- retrieval flexibility
- SQL safety
- follow-up context support
- cloud deployment
- monitoring
- CI and tests

Main area still needing work:

- latency optimization, especially for mixed end-to-end flows

That is a realistic and healthy conclusion.

Not every deployed AI system is “finished.”

This project already proves:

- it works
- it is grounded
- it is deployable
- it is testable
- it is explainable

---

38. Final Learning Summary

If you remember only a few things from this project, remember these:

1. RAG alone is not enough for many enterprise AI problems.
2. Routing between RAG, SQL, and hybrid matters a lot.
3. Retrieval strategy should depend on the question.
4. Advanced features like HyDE, CRAG, and SRAG should be used intentionally.
5. SQL safety and approval are major enterprise requirements.
6. Good AI products need sources, metadata, and observability.
7. Production validation includes CI, evals, logs, dashboards, and load tests.

This project is valuable because it teaches both:

- AI application design
- real software engineering around AI

---

39. Suggested Next Reading

- PRODUCT_REQUIREMENTS_DOCUMENT.md
- WRITEUP.md
- AWS_DEPLOYMENT.md
- REBUILD_FROM_SCRATCH_GUIDE.md
- loadtests/results/FINAL_LOADTEST_RESULTS_20260708.md

---

40. Questions And Answers (50)

Below are 50 questions and answers about this project, ordered from basic to advanced topics. The answers are intentionally more detailed and point back to the real implementation files and behaviors in `new-adv-rag-implement`.

Basic Questions

1. What is `new-adv-rag-implement`?

It is a production-minded AI assistant project that combines Retrieval-Augmented Generation, Text-to-SQL, hybrid routing, human approval for SQL, AWS deployment, CloudWatch monitoring, testing, and CI. In practical terms, it is designed to answer three kinds of questions: document questions, database questions, and mixed questions that need both. You can see this scope reflected across README.md, WRITEUP.md, and the actual routing/orchestration code in app/services/router_service.py and app/core/graph.py.

2. What does RAG mean in this project?
RAG stands for Retrieval-Augmented Generation. In general, RAG means the model answers using retrieved external context instead of relying only on its own parametric memory. In this project, the flow is: retrieve chunks from Qdrant, optionally improve them with reranking/HyDE/CRAG/SRAG, build spotlighted context, and then generate the answer. That behavior is implemented in app/services/rag_service.py. A concrete example is the question: "What does our escalation policy say for P1 incidents?" The system retrieves policy chunks, sends them to the LLM, and returns an answer with sources and chunk previews.

3. Why is this project more than a basic chatbot?
Because it does not follow the simple pattern of "user asks question, LLM answers." Instead, it performs auth, rate limiting, token budgeting, input safety checks, context rewriting, route selection, retrieval strategy selection, SQL validation, SQL approval, caching, output validation, and monitoring. That is why the project feels more like a backend system than a demo. Evidence for this exists in the API layer app/api/query.py, the security modules under `app/security/`, and the graph orchestration in app/core/graph.py.

4. What are the three main query routes?
The three routes are `rag`, `sql`, and `hybrid`. `rag` means the answer comes mainly from retrieved documents. `sql` means the answer comes mainly from PostgreSQL through generated SQL. `hybrid` means the system uses both SQL results and RAG context in one response. The route decision is made in app/services/router_service.py, and the graph branches are defined in app/core/graph.py.

5. What is a `rag` route in detail?
A `rag` route is used when the question is best answered from documents rather than structured tables. In general, that means retrieving chunks, building context, and asking the LLM to answer based on those chunks. In this project, the RAG route also supports advanced retrieval features such as reranking, HyDE, CRAG, and self-reflection. A good example is: "What is the difference between a ConfigMap and a Secret in Kubernetes?" The answer should come from Kubernetes corpus files, not from a database. The resulting metadata usually shows `route="rag"` plus chunk previews and source names.

6. What is a `sql` route in detail?
A `sql` route is used when the question is really asking for a structured lookup, count, aggregate, or filter over rows. In general, this is the right path for metrics or business data questions. In this project, the system generates SQL, validates it with `sqlglot`, and can stop for approval before executing it. An example is: "How many P1 incidents were resolved last quarter?" That should not be answered by retrieving text chunks alone, because the true answer belongs in structured incident data. The logic for SQL generation and execution is in app/services/sql_service.py, while the formatting logic is in app/core/graph.py.

7. What is a `hybrid` route in detail?
A `hybrid` route is used when one part of the answer belongs in a database and another part belongs in documents. In general, this is one of the most valuable enterprise AI patterns because many real questions mix facts with policy or operational guidance. In this project, the graph retrieves RAG context first, then generates SQL, then after approval executes SQL and asks the model to combine both sides. Example question: "How many P1 incidents were resolved last quarter and what does our escalation policy say for P1 incidents?" The SQL gives the count, while the RAG path gives the policy explanation. That combined synthesis is implemented in the `run_hybrid` and `build_hybrid_execution_result` logic in app/core/graph.py.

8. Why does this project use PostgreSQL?
PostgreSQL is used because this project needed a real relational database for users and structured operational data. In general, if you want safe Text-to-SQL and hybrid reasoning, you need a structured data store. In this implementation, PostgreSQL supports auth-related storage and operational queries such as incident counts. It is also realistic from a portfolio perspective because enterprise systems usually combine unstructured and structured data rather than using vectors alone.

9. Why does this project use Qdrant?
Qdrant is the vector database that stores embeddings for document chunks. In general, vector databases allow semantic retrieval by finding chunks with embeddings close to the query embedding. In this project, Qdrant is used through the vector-store layer for dense, sparse, and hybrid retrieval. The integration path is visible in app/services/vector_store.py and in ingestion through app/services/ingestion_service.py.

10. Why does this project use Streamlit?
Streamlit was chosen to provide a fast interactive control panel for the AI system. In general, Streamlit is useful when you want to build a practical UI quickly and focus on backend behavior. In this project, Streamlit shows presets, manual vs auto retrieval controls, Active Config, sources, chunks, raw JSON, recent query history, admin tools, and SQL approval buttons. You can see this full UI behavior in scripts/streamlit_app.py.

Retrieval Questions

11. What is dense retrieval?
Dense retrieval uses embeddings, which are numerical vector representations of text. In general, it is good when the user's question is semantically related to the source text even if the wording is different. In this project, dense retrieval is used when the question is conceptual or paraphrased. For example, the question "How do I make sure my app keeps running even if a server dies?" may not contain exact words like "replicas" or "high availability," but dense retrieval can still find relevant Kubernetes material. Dense behavior is used in app/services/rag_service.py and can be auto-selected by app/services/auto_retrieval.py.

12. What is sparse retrieval?
Sparse retrieval is lexical, meaning it cares more about exact tokens and matching terms. In general, it is important when a question contains very specific technical strings. In this project, sparse retrieval helps rescue exact operational matches such as `ErrImagePull`, `ImagePullBackOff`, or `OOMKilled`. The implementation evidence is not only in the retrieval code but also in tests such as `tests/test_vector_store.py` and `tests/test_reranking.py`, which specifically verify exact troubleshooting term handling.

13. What is hybrid retrieval?
Hybrid retrieval combines dense and sparse retrieval. In general, this is often better for enterprise corpora because some documents match by meaning and some by exact phrasing. In this project, hybrid retrieval became a very strong default because Kubernetes and operations questions often contain both conceptual and exact terms. For example, "What is the difference between a ConfigMap and a Secret in Kubernetes?" benefits from semantic understanding of the concept plus lexical grounding on the object names `ConfigMap` and `Secret`.

14. Why did hybrid retrieval become an important default here?
Because the corpus and the user questions are mixed. Some questions behave like policy lookups, some like troubleshooting, some like conceptual comparisons. Hybrid retrieval is the safest middle ground. This conclusion is supported by the evaluation writeups and by the UI guidance in scripts/streamlit_app.py, where `hybrid + rerank` is described as the best manual default. The eval summaries also pointed to hybrid-style profiles as strong practical choices.

15. What is reranking and what is implemented here?
In general, reranking means: first retrieve a set of candidate chunks, then rescore them with a more precise relevance model. The goal is to improve the final ordering because the first retrieval step can include partially relevant or noisy chunks. In this project, reranking is integrated into the RAG pipeline in app/services/rag_service.py using a `Reranker`. A practical example is a ConfigMap/Secret question where multiple Kubernetes docs are retrieved. Reranking helps put the direct comparison chunks ahead of less relevant dashboard or catalog chunks. Evidence for this appears in tests such as `tests/test_rag_source_filtering.py`, especially the cases that drop unrelated chunks and negative rerank tails.

16. What is HyDE in general and what is implemented here?
HyDE stands for Hypothetical Document Embeddings. In general, the idea is that the system first generates a short plausible answer, then uses that synthetic text to retrieve better source documents. That helps when the user question is vague or conceptual. In this project, HyDE is implemented in app/services/hyde.py. The `HyDERetriever` first keeps the original question, then generates one or more hypothetical answers, normalizes them, embeds them, retrieves results for each, and fuses them. Example question: "How do I make sure my app keeps running even if a server dies?" A plain lexical search may miss the best chunks, but HyDE can generate answer-like text about replicas, rescheduling, or availability, which leads retrieval closer to the right documents. That is why the "HyDE Search" preset exists in the frontend.

17. What is CRAG in general and what is implemented here?
In general, CRAG is a corrective retrieval idea: do not assume the retrieved context is good enough just because retrieval returned something. Instead, grade the relevance of the retrieved context and decide whether to trust it, retry locally, or fall back. In this project, CRAG is implemented in app/services/crag.py. The `grade_chunks` function takes the user question plus the top retrieved chunks and asks a grader model to return JSON with `relevance_score`, `relevance_label`, `confidence`, and `reasoning`. Then `crag_pipeline` uses thresholds from settings to decide what to do next. If the score is high enough, it accepts the chunks. If the score is in an ambiguous range, it can retry retrieval locally, for example by forcing HyDE or retrieving more. If the score is very low and web fallback is allowed, it can fall back to web search.
Example: imagine the question is about ConfigMap vs Secret, but the initial retrieval also includes irrelevant dashboard documentation. CRAG grades the retrieved set. If the quality looks weak, the system can try a corrective retrieval step rather than blindly answer from noisy evidence. In app/services/rag_service.py, the returned metadata exposes `crag_score`, `crag_label`, and whether web fallback was used. The frontend then surfaces these values in the metadata area so the user can see that retrieval quality was evaluated.

18. What is SRAG in general and what is implemented here?
SRAG means self-reflective RAG. In general, it adds a review step after an initial answer is drafted. Instead of always trusting the first

answer, the system checks whether the answer is vague, incomplete, unsupported, or whether a sharper question would help retrieval. In this project, reflection is implemented in app/services/self_reflective.py. The `reflect_on_answer` function asks a grader model to return `reflection_score`, `needs_regeneration`, `refined_question`, and `reasoning`. Then `should_regenerate` decides whether to re-run the answer step.
Example question: "My Kubernetes app keeps restarting, and I am not sure whether the root cause is the liveness probe, OOMKilled, or a bad image. How should I troubleshoot it step by step?" This is exactly the kind of question where the first draft may be too shallow. Reflection can decide that the answer needs regeneration or that the question should be refined for better retrieval. In app/services/rag_service.py, the system tracks `reflection_iterations`, `reflection_score`, and `refined_question`, and those values are returned in the metadata.

19. When is HyDE useful in this project?
HyDE is most useful when the question is semantic, conceptual, or paraphrased rather than exact. In auto mode, this is why conceptual questions often resolve to dense retrieval with HyDE turned on. A good example is: "How do I make sure my app keeps running even if a server dies?" The user may not know the exact Kubernetes vocabulary, but HyDE can invent helpful terms in the hypothetical answer and improve retrieval. This behavior is reflected in the `HyDE Search` preset in scripts/streamlit_app.py.

20. When is SRAG useful in this project?
SRAG is most useful when the user is uncertain, when the problem is ambiguous, or when multiple root causes are possible. The explicit example used in this project is the restarting Kubernetes app question with possible liveness probe, OOMKilled, or bad image causes. The auto policy in app/services/auto_retrieval.py contains a reflection-specific branch for ambiguous troubleshooting questions and enables SRAG there. That was later reinforced with the regression test `test_auto_retrieval_enables_srag_for_ambiguous_troubleshooting_question` in `tests/test_auto_retrieval.py`.

Routing And Auto Mode Questions

21. What is intent routing?
Intent routing is the step where the backend decides what kind of question it is dealing with. In general, an AI system often performs better if it first understands whether the user needs retrieval, SQL, or both. In this project, the router chooses among `rag`, `sql`, and `hybrid`. This decision controls which graph path runs next. Without routing, the model might try to answer a count question from documents alone or a policy question from SQL alone, both of which would be weaker.

22. Where is intent routing implemented?
It is implemented in app/services/router_service.py. That file contains SQL hint patterns, SQL entity patterns, RAG hints, and hybrid connectors. The logic is intentionally heuristic and transparent, which makes it easier for a learner to understand and for the developer to debug.

23. How does the router decide between RAG and SQL?
The router looks for SQL-style structures such as `how many`, `count`, `average`, plus entities like `users`, `incidents`, `cluster`, or `SLA`. If it sees both a SQL hint and a SQL entity, it leans toward SQL. If it also sees RAG hints like `policy`, `runbook`, or `Kubernetes`, then it can choose hybrid instead. For example, "How many P1 incidents were resolved last quarter?" becomes `sql`, while "How many P1 incidents were resolved last quarter and what does our escalation policy say?" becomes `hybrid`. This behavior is covered in `tests/test_hybrid_routing.py`.

24. What is `search_mode="auto"`?
`search_mode="auto"` means the frontend is not explicitly forcing `dense`, `sparse`, or `hybrid`. Instead, the backend examines the user question and resolves the best strategy itself. In general, this is useful because most production users should not need to understand retrieval theory. In this project, auto mode still returns its chosen settings in metadata, so the decision is not hidden from the user.

25. Where is auto mode implemented?
It is implemented in app/services/auto_retrieval.py. That file defines `AutoRetrievalDecision`, helper functions for recognizing conceptual, exact-token, troubleshooting, action, and reflection patterns, and `resolve_auto_retrieval`, which returns the final strategy and flags.

26. What signals does auto mode use?
It uses several families of signals. Exact-token signals catch things like `OOMKilled` or `ErrImagePull`. Conceptual signals catch phrases like "best way to think about" or "difference between." Troubleshooting and guidance signals catch terms like `runbook`, `policy`, `fix`, or `debug`. Reflection signals catch uncertainty phrases such as `not sure`, `root cause`, or questions listing multiple causes with `or`. This is visible directly in the constant groups at the top of app/services/auto_retrieval.py.

27. Why did `how do i scale` not activate SRAG?

Because the current auto policy treats that question as an operational action question, not as an ambiguous troubleshooting question. It contains "how do I" and "scale," which match action signals, but it does not contain reflection signals such as "not sure," "root cause," or multiple competing causes. So the backend correctly resolved it to hybrid retrieval with rerank and CRAG, but not SRAG. This exact debugging outcome was validated against the live backend and then the frontend preset was updated so that the SRAG preset uses a truly reflection-worthy question.

28. What metadata proves auto mode worked?

The key metadata fields are `requested_search_mode`, `resolved_search_mode`, `resolved_enable_rerank`, `resolved_enable_hyde`, `resolved_enable_crag`, `resolved_enable_self_reflective`, `auto_enabled`, `auto_strategy_reason`, and `auto_applied_flags`. These fields are generated by the auto decision object and merged into the response metadata in app/services/rag_service.py. The frontend reads these values and shows them in the Active Config panel.

29. Why keep Manual mode if Auto mode exists?

Manual mode is still important because it helps with education, demos, evaluation, and debugging. If a recruiter, hiring manager, or teammate wants to see how dense-only retrieval behaves versus hybrid + rerank, Manual mode makes that visible. It also helps isolate whether a problem comes from retrieval strategy or from the corpus itself. In this project, Manual mode is explicitly exposed in the Streamlit query interface in scripts/streamlit_app.py.

30. What is the best manual default in this project?

The best manual default is `hybrid + rerank`. This conclusion came from the project's practical evaluation and retrieval-tuning work. Hybrid gives both semantic and lexical support, and reranking improves the final chunk order. That is why the frontend guidance text says "hybrid + rerank is the best manual default." It is not just a slogan; it reflects the way the mixed enterprise corpus behaved in practice.

Ingestion And Data Questions

31. What happens during document ingestion?

During ingestion, the system computes a content hash, checks whether that content has already been indexed, saves the original file to storage, parses the document, creates chunks, embeds those chunks, upserts them into Qdrant, and stores metadata. In this project, that is centralized in app/services/ingestion_service.py. This design is important because it avoids putting ingestion logic directly inside routes or ad hoc scripts.

32. Why is a content hash used during ingestion?

A content hash is used to detect duplicates based on the actual file contents, not only on the filename. In general, this is much safer because users may upload the same document under different names. In this project, the ingestion service computes a hash through the document cache service and uses it as the basis for dedupe. The behavior is covered by `tests/test_ingestion_service.py`, which verifies that ingesting the same file twice skips the second indexing operation.

33. Why are documents chunked?

Documents are chunked because whole documents are usually too large and too unfocused to send directly to the LLM during each query. Chunking creates smaller retrieval units that are easier to rank and pass into prompts. In this project, chunking is part of the document processing flow before vectors are stored. For example, a long Kubernetes manual might contain sections on ConfigMaps, Secrets, probes, RBAC, and services. Chunking allows the system to retrieve only the relevant parts for each question.

34. Why are original files stored in addition to vectors?

Vectors are great for retrieval, but they do not replace the original file. The original file is still useful for auditability, future reprocessing, and traceability. In this project, the ingestion service stores the original file via the storage backend and records metadata such as `storage_backend`, `storage_key`, and `storage_url`. This also makes the system more enterprise-like because source preservation matters.

35. Where is ingestion logic centralized and why?

It is centralized in app/services/ingestion_service.py. Centralization matters because ingestion is a multi-step workflow: dedupe, storage, parsing, chunking, embedding, vector upsert, and metadata persistence. If that logic were scattered across routes and scripts, it would be harder to test, explain, and maintain.

SQL And Safety Questions

36. What is Text-to-SQL in this project?

Text-to-SQL means converting a natural-language question into SQL so a relational database can answer it. In general, it is useful when users want counts, aggregates, filters, or lists over structured data. In this project, a question like "How many P1 incidents were resolved

last quarter?" is a natural Text-to-SQL candidate. The SQL generation and validation logic live in app/services/sql_service.py, while the graph handles approval and execution flow in app/core/graph.py.

37. Why is SQL treated as higher risk than RAG?
Because SQL acts directly on structured data systems. Even if the intent is read-only, a badly designed SQL path could be unsafe or misleading. In contrast, a weak RAG answer is usually an answer-quality problem, while bad SQL can become a data-governance problem. That is why this project treats SQL with more control layers: route separation, AST validation, caching, and approval flow.

38. How does the project make SQL safer?
It uses `sqlglot` to parse and validate SQL structurally, not just by string matching. In general, this is much stronger than checking whether the query contains the word `DROP`. The SQL service blocks unsafe patterns such as destructive statements, multi-statement SQL, and disallowed forms. This is supported by tests in `tests/test_sql_service.py`, which verify that SELECT-style queries are allowed while `DROP`, `EXPLAIN`, multi-statement SQL, and `SELECT INTO` are blocked.

39. What is human-in-the-loop SQL approval in detail?
Human-in-the-loop approval means the system can pause before executing SQL and require explicit user confirmation. In this project, LangGraph's `interrupt` mechanism is used in app/core/graph.py. The user then approves through `/query/sql/execute`. This is surfaced in the frontend by showing the SQL, its explanation, and "Approve" / "Reject" buttons. That means the system is not blindly autonomous on structured data access.

40. What happens if SQL approval context is lost after a restart?
If the in-memory graph state is gone, the system now returns an explicit `409` response explaining that the approval context is missing. This is a meaningful production hardening step because otherwise the system might fail silently or behave confusingly. The query API code handles this clearly in app/api/query.py, and tests such as `test_query_sql_execute_returns_explicit_missing_context` verify the behavior.

Frontend And UX Questions

41. What does the Active Config section in the UI show?
The Active Config section shows the retrieval settings that are actually in effect for the current query. That includes Experience Mode, Search Mode, Top K, Rerank, CRAG, HyDE, and SRAG. In Auto mode, these values may come from backend-resolved metadata rather than from manual button choices. This is implemented in scripts/streamlit_app.py and is useful because it makes backend behavior transparent.

42. Why is the Active Config section important?
It is important because AI systems are much easier to trust when users can see what strategy actually ran. Without Active Config, a user might think they are seeing "manual hybrid" behavior when the backend actually resolved to "auto dense + HyDE," or vice versa. This section makes the system explainable and easier to debug.

43. What do the Sources and Chunks tabs help with?
The Sources tab shows where the answer came from at the source-file or URL level. The Chunks tab shows the actual retrieved text snippets and their scores. In general, this is crucial for grounded AI because it gives the user a way to inspect whether the evidence is relevant. In this project, the chunk tab can show `score`, `retrieval_score`, and `rerank_score`, which is especially useful when evaluating retrieval quality or noisy-chunk filtering behavior.

44. What does the Raw JSON tab help with?
The Raw JSON tab helps because it exposes the entire backend response, including metadata that the UI may summarize elsewhere. This is valuable for debugging and learning. For example, if you want to confirm whether auto mode chose SRAG, or whether CRAG labeled the retrieval as `somewhat_relevant`, the Raw JSON view makes that visible without guessing.

Deployment, Testing, And Operations Questions

45. Why was AWS ECS Fargate chosen for deployment?
ECS Fargate was chosen because it offers a production cloud runtime without requiring the project to manage EC2 servers or a Kubernetes cluster for the app itself. In general, that makes it a good balance between realism and operational simplicity. Since this project already focuses on Kubernetes knowledge as content, using ECS Fargate for hosting kept the deployment manageable while still demonstrating cloud architecture skill. The deployment details are documented in AWS_DEPLOYMENT.md.

46. What are the main AWS services used in production, and what does each do?
The main AWS services are ECS Fargate, ECR, ALB, Route 53, ACM, RDS PostgreSQL, S3, Secrets Manager, and CloudWatch. ECS runs the backend and Streamlit containers. ECR stores images. ALB routes traffic. Route 53 and ACM provide DNS and HTTPS. RDS hosts

PostgreSQL. S3 stores original uploaded files. Secrets Manager stores sensitive config. CloudWatch handles logs, alarms, metrics, and dashboards. These roles are described in AWS_DEPLOYMENT.md and surfaced in the deployment info now shown in the frontend.

47. What does the CloudWatch dashboard prove?
The CloudWatch dashboard proves that the deployed system is observable. It consolidates alarm state, ALB response time, ECS CPU/memory, RDS metrics, request volume, target health, and backend request logs. During load tests, it provides real evidence that the backend is receiving traffic, the targets remain healthy, and the latency/CPU patterns change during the run. This is why the project built `new-adv-rag-ops` and captured dashboard screenshots during testing.

48. What did the load tests show overall?
The load tests showed that health, RAG, SQL approval, and hybrid flows worked end to end without HTTP failures in the validated runs. The health test was fast and clean. The RAG test passed functionally and passed its p95 threshold on a repeated run. The mixed smoke test passed functionally but exceeded the p95 latency threshold. That means correctness and availability were strong, while latency optimization remains the main next improvement area. The evidence is summarized in loadtests/results/FINAL_LOADTEST_RESULTS_20260708.md.

49. What is the role of GitHub Actions in this project?
GitHub Actions provides continuous integration. It automatically runs the project's tests and eval-smoke jobs on push, which helps prevent regressions. In this project, CI also exposed a hidden dependency: some tests originally depended on a locally available Postgres-backed pending SQL store. That passed locally but failed in GitHub Actions. Fixing that made the repo healthier and the tests more portable. This is a real engineering lesson, not just a tooling detail.

50. What is the biggest lesson from this project?
The biggest lesson is that a strong AI project is not just "call an LLM and return text." A serious AI system needs correct routing, retrieval quality controls, safety layers, explainability, deployment discipline, monitoring, tests, evals, and production validation. This project demonstrates that those pieces can work together in one implementation. That is why it is valuable both as an AI learning project and as a portfolio project for hiring managers.

---

41. 25 Coding Questions

Below are 25 coding-focused interview questions and answers based on the actual implementation in `new-adv-rag-implement`.

1. How did you implement reranking in this project?

Reranking is implemented in app/services/reranking.py. The core class is `Reranker`, and it uses a `CrossEncoder` from `sentence_transformers`.

The flow is:

1. build `(query, chunk.text)` pairs
2. run `CrossEncoder.predict()` to get rerank scores
3. collect original retrieval scores
4. calculate keyword-overlap scores
5. add exact-token bonuses for troubleshooting queries
6. normalize and blend those signals into one final score
7. sort descending and keep top K

This is important because I did not use the cross-encoder score alone. I blended:

- rerank score
- original retrieval score
- keyword overlap
- troubleshooting bonuses

That made the reranker more robust for operational questions like `ErrImagePull`, where lexical precision matters.

2. Why did you use a CrossEncoder for reranking instead of only vector similarity?

Vector similarity is good for first-pass retrieval, but it can still produce noisy results. A cross-encoder is stronger for pairwise relevance judgment because it looks at the query and the candidate chunk together.

In this project, vector retrieval gets candidate chunks first. Then the cross-encoder helps answer a more precise question:

> "Given this exact query, how relevant is this exact chunk?"

That is especially useful for:

- concept comparisons
- troubleshooting queries
- mixed corpora with distractor chunks

The code evidence is in app/services/reranking.py.

3. How exactly is the rerank score calculated?

The blended score is not a single model output. In the current implementation, the final score is:

- `0.4 * normalized cross-encoder score`
- `0.3 * normalized retrieval score`
- `0.3 * keyword overlap`
- plus exact-token troubleshooting bonuses

That formula appears directly in the `blended_score` calculation inside `Reranker.rerank`.

Why blend instead of trusting one number?

Because:

- raw retrieval score is still useful
- lexical overlap helps preserve exact operational matches
- cross-encoder adds a stronger relevance judgment

This is a practical coding decision, not only a theoretical one.

4. How did you preserve exact troubleshooting terms during reranking?

I added `_exact_token_bonus()` and `_TERM_EXPANSIONS` in app/services/reranking.py.

That logic gives extra credit when the chunk matches strong troubleshooting terms such as:

- `errimagepull`
- `imagepullbackoff`
- `oomkilled`
- `crashloopbackoff`

It also rewards chunks whose source names look like runbooks or troubleshooting docs.

This prevents the reranker from accidentally pushing down highly relevant operational chunks just because another chunk is semantically broader.

5. How did you implement HyDE in code?

HyDE is implemented in app/services/hyde.py through the `HyDERetriever` class.

The implementation does this:

1. start with the original question
2. generate one or more hypothetical answer-like texts using the LLM
3. normalize and deduplicate those generated texts
4. retrieve results for each generated query

5. fuse the result lists using reciprocal-rank fusion

This is a clean implementation because HyDE is treated as a retrieval strategy layer, not as random prompt logic embedded in the route.

6. How did you implement CRAG in code?

CRAG is implemented in app/services/crag.py.

The key functions are:

- `grade_chunks()`
- `crag_pipeline()`

`grade_chunks()` sends the question plus retrieved chunks to a grader model and asks for strict JSON output with:

- `relevance_score`
- `relevance_label`
- `confidence`
- `reasoning`

Then `crag_pipeline()` decides:

- accept the current chunks
- retry retrieval locally
- or fall back to web if allowed and relevance is too low

This is an example of coding retrieval correction as a structured pipeline instead of a vague concept.

7. How did you implement self-reflection in code?

Self-reflection is implemented in app/services/self_reflective.py.

The process is:

1. generate an initial answer
2. pass question, answer, and context to a grader model
3. ask the grader for:
    - reflection score
    - whether regeneration is needed
    - a refined question
4. if needed, regenerate using the refined question

In app/services/rag_service.py, this is controlled with:

- `reflection_iterations`
- `reflection_score`
- `refined_question`

8. How is auto retrieval implemented at the code level?

Auto retrieval is implemented in app/services/auto_retrieval.py.

The file defines:

- several signal groups like troubleshooting, conceptual, action, and reflection hints
- helper functions such as `_is_vague_query`, `_has_exact_token_signal`, and `_has_reflection_signal`
- a dataclass `AutoRetrievalDecision`
- the decision function `resolve_auto_retrieval()`

This is good coding structure because the behavior is:

- readable

- testable
- explicit

The test coverage for this logic lives in `tests/test_auto_retrieval.py`.

9. Why did you use a dataclass for auto retrieval decisions?

I used `AutoRetrievalDecision` so the result is explicit and typed instead of returning a loose dictionary everywhere.

That dataclass groups together:

- requested mode
- resolved mode
- resolved flags
- auto reason
- applied flags

It also contains helper methods for:

- cache context
- graph flags
- metadata

This makes the code cleaner because the same decision object can feed multiple layers without repeating transformation logic.

10. How does the RAG pipeline actually run in code?

The main flow is inside `_run_rag_impl()` in app/services/rag_service.py.

The steps are:

1. resolve auto retrieval if needed
2. build cache context
3. try query cache
4. retrieve chunks
5. trim troubleshooting noise
6. run CRAG pipeline
7. normalize temp sources
8. prefer operational, canonical, and anchor-matched sources
9. drop noisy dashboard and negative-tail chunks
10. build spotlighted context
11. generate answer
12. optionally reflect/regenerate
13. compute confidence
14. build metadata and cache the result

This is one of the strongest implementations in the repo because it turns many retrieval ideas into one traceable function.

11. How did you reduce noisy chunks in code?

Noise reduction happens in multiple helper functions inside app/services/rag_service.py, such as:

- `_trim_troubleshooting_noise`
- `_prefer_operational_sources`
- `_prefer_entity_named_sources`
- `_prefer_anchor_matched_sources`
- `_drop_non_kubernetes_corpus_noise`
- `_drop_dashboard_catalog_noise`
- `_drop_negative_rerank_tail`

This is important because “better prompting” alone does not solve noisy evidence. The retrieval set itself must be cleaned before generation.

12. How does the system rewrite follow-up questions?

Follow-up rewriting is implemented in app/services/conversation_context.py.

The code:

1. detects known subjects such as `ConfigMap`, `Secret`, `Pod`, `ServiceAccount`, and `RBAC`
2. checks whether the current question looks like a follow-up
3. looks at the last conversation turn
4. resolves ambiguous references like:
   - “the non-sensitive one”
   - “that same object”
   - “if I mounted it”
5. rewrites the question into a clearer form

Example:

- “Would you use that same object for a database password?”

can become:

- “Would you use a Secret for a database password?”

13. How is SQL validation implemented?

SQL validation is implemented in app/services/sql_service.py, mainly through `is_select_only()`.

The code:

1. parses SQL with `sqlglot`
2. rejects if parsing fails
3. rejects multiple statements
4. rejects statements that are not query-like
5. rejects forbidden AST node types like:
   - `Drop`
   - `Delete`
   - `Update`
   - `Insert`
   - `Create`
   - `Alter`

This is significantly stronger than regex filtering.

14. How did you handle relative-time SQL questions?

Relative-time logic is handled in app/services/sql_service.py using:

- `_question_uses_relative_time()`
- `_sql_uses_dataset_max_anchor()`
- `_sql_uses_runtime_time_anchor()`

The service explicitly pushes the LLM to use `CURRENT_DATE` or similar runtime anchors for terms like:

- today
- yesterday
- last week
- last quarter

This matters because without enforcement, a model might incorrectly anchor to `MAX(date_column)` in the table instead of real runtime time semantics.

15. How did you implement heuristic SQL shortcuts?

Inside app/services/sql_service.py, `_heuristic_sql()` handles a set of common demo/operational questions with deterministic SQL.

This is useful because:

- it reduces LLM cost
- improves predictability
- avoids unnecessary model calls for known patterns

Examples handled include:

- resolved P1 incidents last quarter
- most-paged on-call engineer this month
- open P2 alerts
- cluster detail lookups

16. How does hybrid execution work in code?

Hybrid execution is handled in app/core/graph.py.

The main flow is:

1. retrieve RAG context via `retrieve_rag_for_hybrid`
2. generate SQL
3. pause for approval
4. run SQL
5. combine document context and SQL results
6. generate final hybrid answer

The actual answer merge uses:

- document context
- SQL summary
- SQL rows

and passes them to the LLM with a system prompt that explicitly tells it to stay grounded.

17. How did you format SQL answers into natural language?

The SQL answer formatting logic is in `_format_sql_answer()` in app/core/graph.py.

Instead of always returning raw JSON rows, the code tries to produce human-readable responses like:

- “50 P1 incidents were resolved last quarter.”
- “The query returned one row: …”
- “No matching records were found.”

This improves user experience and is covered by tests such as `tests/test_graph_sql_formatting.py` and `tests/test_sql_response_format.py`.

18. How did you implement pending SQL approvals?

Pending approvals are implemented through two layers:

1. LangGraph interrupt/resume in app/core/graph.py
2. persistence and resume logic in app/api/query.py and app/services/pending_sql_store.py

When SQL approval is needed, the system returns a `pending_sql` block with:

- SQL text

- explanation
- query ID

The user then approves through `/query/sql/execute`.

19. How is caching implemented in code?

Caching is implemented in app/services/query_cache_service.py.

The service supports multiple namespaces:

- intent
- rag_answer
- sql_gen
- sql_result
- approved_answer
- embedding

It hashes keys with SHA-256 and can use:

- Upstash Redis when configured
- in-memory fallback otherwise

This is a solid coding pattern because the same interface works both locally and in cloud environments.

20. Why did you implement both Redis and in-memory cache fallback?

Because production and local development have different constraints.

In production, Redis is valuable for shared caching. In local development or degraded conditions, in-memory fallback keeps the app functional.

The `_build_redis_client()` and `_disable_redis()` logic in app/services/query_cache_service.py support this behavior.

This is a practical resilience pattern.

21. How is cache correctness preserved across retrieval settings?

The RAG cache key is not just the question text. It also includes a cache context that contains:

- top K
- search mode
- rerank flag
- HyDE flag
- CRAG flag
- self-reflective flag
- schema version

This prevents a cached answer from one retrieval configuration being incorrectly reused for another configuration.

That logic is implemented through `_cache_context()` in app/services/rag_service.py.

22. How did you calculate confidence in the RAG response?

Confidence is calculated in `_calculate_confidence()` in app/services/rag_service.py.

The signal combines:

- top chunk quality
- average quality of top chunks
- retrieval coverage
- CRAG score if enabled
- reflection score if present

This is a heuristic confidence, not a mathematically guaranteed truth score. But it is still useful because it is based on actual retrieval and review signals instead of a hardcoded constant.

23. How did you calculate cost-saved values?

Cost-saved logic is handled by `cost_service` plus response assembly code. The key idea is:

- fresh responses should show zero savings
- cache hits can show avoided LLM and infra cost

The code for cost derivation and formatting appears in app/api/query.py, especially `_derive_cost_components()`, `_display_saved_values()`, and `_approved_cache_payload_from_result()`.

This prevents misleading "saved cost" claims on first-run answers.

24. How did you make the tests independent of local infrastructure?

One important coding fix was making API tests independent of local Postgres-backed pending SQL state. The solution was to use an in-memory fake pending SQL store inside `tests/test_query_api.py` through an `autouse` fixture.

This matters because otherwise tests can pass only on a developer machine with local services running but fail in CI.

This was a real GitHub Actions issue that had to be fixed after push.

25. What coding decisions in this project are most interview-worthy?

The most interview-worthy coding decisions are:

- using a `CrossEncoder` reranker plus lexical bonuses instead of trusting one score
- implementing CRAG as a real pipeline with grading and corrective retry
- implementing SRAG with regeneration control
- keeping auto retrieval as explicit rule-based policy with tests
- separating document dedupe from query-time cache
- validating SQL structurally with `sqlglot`
- using LangGraph interrupts for human approval
- making cache keys retrieval-aware
- cleaning noisy chunks before generation
- hardening tests so they do not depend on hidden local services

These are strong interview topics because they show practical AI engineering, not only framework familiarity.

---

42. Bonus: 18 Trade-off, Failure-Mode, and Scale Questions

The 50 questions in Section 40 and the 25 coding questions in Section 41 mostly answer "how does this work" and "why did you build it this way." That covers a "walk me through your project" interview well, but interviewers often follow up with harder questions: what breaks, what did you cut corners on, and how would this hold up at 10x scale. These 18 questions are meant to prep for that second round. Every technical detail below was verified directly against the code in `new-adv-rag-implement` before writing it down.

1. What happens if the CRAG or SRAG grader model returns malformed JSON?
Both `grade_chunks()` in app/services/crag.py and the reflection grader in `self_reflective.py` call `generate_with_json()` in app/services/llm_service.py, which sets `response_format={"type": "json_object"}` on the OpenAI call. That structured-output mode makes truly malformed JSON rare, but it does not guarantee the JSON contains the right keys or sane values. The parsing itself (`json.loads(result["text"])` in `grade_chunks`) has no try/except around it and no retry — if OpenAI's JSON mode ever returns something `json.loads` can't parse, that call raises and propagates up as an unhandled exception rather than degrading gracefully. Honest answer for an interview: JSON mode covers the common case, but there is no explicit retry-on-parse-failure or schema validation layer, and that's a real gap I'd close with a `try/except` plus one retry before failing the CRAG/SRAG step closed (i.e., skip the corrective step rather than 500 the whole request).

2. How do the unit tests stay deterministic when the whole system is LLM-in-the-loop?
They don't call the LLM at all. Tests use `monkeypatch` to replace the functions that would call OpenAI, not the OpenAI client itself. For example, `tests/test_query_api.py` monkeypatches `query_api.classify_intent`, `query_api.graph.invoke`, and the pending-SQL store functions directly with fake implementations that return fixed values (see the `_fake_pending_sql_store` fixture and calls like `monkeypatch.setattr(query_api, "classify_intent", lambda question: "rag")`). This is a legitimate and common pattern — it means the tests verify orchestration, routing, and API-contract behavior, not model output quality. Model output quality is instead the job of the `Eval/` harness described in Section 32, which is a meaningfully different kind of check. A good interview answer distinguishes these two layers explicitly: unit tests for deterministic control flow, evals for probabilistic answer quality.

3. There's no database connection pooling — what's the actual risk, and how would you fix it?
Both app/services/sql_service.py and app/services/pending_sql_store.py call `psycopg2.connect(settings.database_url)` directly, opening and closing a fresh TCP+auth connection on every SQL execution, approval save, and approval resolve. At low load this is invisible. Under concurrent traffic it adds real per-request latency (TCP handshake, TLS if enabled, Postgres auth) and can exhaust RDS's max connection count faster than a pooled setup would. This is a legitimate scale weakness, and the honest fix is a `psycopg2.pool.ThreadedConnectionPool` or moving to SQLAlchemy with `pool_size` / `max_overflow` tuned to the ECS task's expected concurrency, or PgBouncer in front of RDS. This is a good one to volunteer proactively — it shows you know the difference between "works" and "works at scale."

4. The rate limiter fails open when Redis is unavailable — is that the right default?
Yes, and it's a deliberate choice, not an oversight. In app/middleware/rate_limiter.py, `RateLimiter.is_allowed()` wraps the Upstash Redis sliding-window check (`zremrangebyscore` / `zadd` / `zcard` / `expire`) in a bare `try/except Exception` that returns `(True, self.max_requests, 0)` on any failure — the comment literally says "Fail open if Redis is unavailable." The trade-off: if Redis goes down, rate limiting silently stops enforcing instead of blocking all traffic. For an internal/demo-scale system that's the right call, because failing closed would take down the whole app over a caching dependency. For a public-facing production system handling abuse-prone traffic, you'd want a secondary in-process fallback limiter so you're not fully open during a Redis outage. This is a good "trade-offs are contextual" answer to have ready.

5. The eval "faith" scores differ by only 0.016–0.03 between profiles — how much should that be trusted?
Honestly, not a lot without more rigor. Section 32 records baseline ≈0.598, hybrid ≈0.600, hybrid SQL ≈0.614, and CRAG ≈0.585 faithfulness. The gap between the best and worst of those is about 0.03, and the guide doesn't record a sample size or confidence interval for the smoke-eval run. A spread that small could plausibly be within run-to-run noise for an LLM-graded metric. The right answer in an interview is not to defend the number as proof, but to reframe it correctly: these smoke evals are directional regression signals ("did this change make things obviously worse"), not a rigorous A/B result you'd cite as "hybrid SQL is 2.7% better." If pushed, the improvement I'd make is running each profile over a larger, fixed eval set multiple times and reporting mean ± variance before making a comparative claim.

6. CRAG and SRAG both use an LLM to judge LLM output — what's the failure mode there, and how would you catch it?
The core risk is correlated error: if the grader model shares the same blind spots as the generator model (both are OpenAI models with `temperature=0.0` per `grade_chunks()`), it can rate a subtly wrong or unsupported answer as confidently correct — the classic "judge agrees with itself" failure. It's also non-deterministic across model versions even at `temperature=0.0`, since token-level determinism isn't guaranteed by the API. The way to catch this in this project's structure is what `tests/test_rag_source_filtering.py` and the `Eval/` harness already do in spirit — checking retrieval and grounding behavior against fixed expected outcomes rather than trusting the grader's self-report alone. A stronger mitigation I'd propose: periodically sample CRAG/SRAG decisions and have a human or a different model family spot-check agreement, since a single-model judge is the weakest link in the corrective loop.

7. How would this system behave at 10x the current document volume?
Retrieval itself would likely hold up reasonably well — Qdrant is a purpose-built ANN vector store (see app/services/vector_store.py, which configures collections with `Distance.COSINE`) and is designed to scale sub-linearly with corpus size. The more likely bottlenecks are downstream of retrieval: reranking cost grows linearly with `top_k` candidates per query since the `CrossEncoder` scores every `(query, chunk)` pair on the request path (see coding Q1–3), and ingestion throughput would be gated by the embedding API rate limits and the single-threaded per-file flow in app/services/ingestion_service.py unless batched. The honest answer: retrieval scales, reranking and ingestion don't scale for free, and I'd profile those two before assuming the whole pipeline scales linearly.

8. Cross-encoder reranking adds a model call on every request — how would you cut that latency in production?
A few concrete levers, in rough order of effort: (1) reduce the candidate set size fed into `Reranker.rerank()` so the cross-encoder scores fewer pairs per query; (2) batch the `CrossEncoder.predict()` call, which `sentence_transformers` already supports, rather than scoring one pair at a time; (3) move from a full cross-encoder to a smaller distilled reranker or an ONNX-exported version for faster CPU/GPU inference; (4) cache rerank results the same way `rag_answer` caching already works in app/services/query_cache_service.py,

keyed on the same retrieval-aware cache context described in coding Q21. This directly ties to the load-test finding in Section 29 that the mixed end-to-end flow was the slowest path — reranking is one of the concrete places that latency budget is likely going.

9. This is currently single-tenant — what would multi-tenancy require?
Every layer would need a tenant/org identifier threaded through it: Qdrant collections or payload-filtered search scoped per tenant (or one collection with a mandatory `tenant_id` filter on every query), Postgres rows scoped with a `tenant_id` column and row-level security or query-level filtering, cache keys in app/services/query_cache_service.py namespaced by tenant so one tenant's cached answer can never leak to another, and the JWT auth layer extended to carry tenant claims. The riskiest part isn't the SQL or RAG logic — it's making sure retrieval and caching can never cross tenant boundaries, since a cache or vector-search bug there is a data leak, not just a bug.

10. The moderation/redaction layer only regexes emails and phone numbers — what's missing, and why might that matter?
app/security/content_moderation.py redacts with two compiled patterns — `_EMAIL_RE` and `_PHONE_RE` — substituting `[REDACTED_EMAIL]` and similar tags. That catches two common PII shapes but nothing else: no names, physical addresses, SSNs/national IDs, credit card numbers, or IP addresses. Regex-based redaction is also inherently brittle — it catches well-formatted matches and misses anything slightly off-pattern. For a real enterprise deployment handling sensitive input, the honest next step is either a dedicated PII-detection model/service (e.g., Presidio-style NER-based detection) or, at minimum, expanding the regex set and adding test coverage for the gaps. This is a good one to bring up yourself — it shows awareness that "we have redaction" and "we have adequate redaction" are different claims.

11. Chunking is a fixed 1000-character window — what's the trade-off against semantic/structural chunking?
app/services/document_processor.py defines `_CHUNK_SIZE = 1000` with a fixed overlap and slides a character window across the document (`start += _CHUNK_SIZE - _CHUNK_OVERLAP`). This is simple, predictable, and fast, but it's blind to document structure — it can split a ConfigMap explanation mid-sentence or mid-example if the boundary happens to fall there, which is exactly the kind of noisy-chunk problem that app/services/rag_service.py's `_trim_troubleshooting_noise` and related filtering helpers (coding Q11) exist partly to clean up after the fact. Semantic or markdown-header-aware chunking would reduce the amount of noise-filtering needed downstream, at the cost of more complex, format-specific ingestion logic. Fixed-size chunking is a defensible starting point; the noise-filtering layer is effectively compensating for its blind spots.

12. What happens if two requests try to approve/resolve the same pending SQL `query_id` at the same time?
`PendingSQLStore.resolve()` in app/services/pending_sql_store.py runs a plain `UPDATE ... SET status = %s WHERE query_id = %s` with no optimistic-concurrency check (no `WHERE status = 'pending'` guard, no row versioning). Two concurrent approval requests for the same `query_id` could both succeed at the database level, and whichever `resolve()` call's SQL execution completes could end up executing structured, potentially data-touching SQL twice. This is a real edge case that isn't covered by the current test suite. The fix is small — add `WHERE status = 'pending'` to the `UPDATE` and check `cur.rowcount` to detect whether this call actually won the race — but it's worth being able to name unprompted, since "what about concurrent approvals" is an obvious question for a human-in-the-loop SQL flow.

13. If you had to name the single biggest point of fragility in this architecture, what would it be?
The pending SQL approval store, because it sits at the intersection of the two riskiest properties in the system: it's the one place a race condition can lead to duplicate execution of write-adjacent SQL (Q12), and it's Postgres-backed specifically because the earlier in-memory version broke across process restarts and in CI (Section 27/49, coding Q24). It has already had one real production-hardening pass and still has an unaddressed concurrency gap. That combination — "already fixed once, still has a known gap" — is a genuinely strong, specific answer, much stronger than a vague "the LLM could hallucinate" answer that doesn't point at actual code.

14. How would you roll out a retrieval-strategy change (e.g., new rerank weights) without a full deploy?
Today, changing the `0.4 / 0.3 / 0.3` blend in `Reranker.rerank()` (coding Q3) or the auto-retrieval signal thresholds in app/services/auto_retrieval.py means editing code and redeploying the ECS service — there's no feature-flag or config-driven toggle for these values. A safer rollout path would move these weights and thresholds into `app/config.py` (`Settings`) so they're env-var or Secrets-Manager driven, then run the `Eval/` harness against both the old and new values before flipping traffic, ideally with the CloudWatch dashboard (Section 28) watched during the switch. This ties directly into the "hybrid + rerank is the best default" claim in Section 30 — that conclusion should be re-validated any time these constants change, not treated as permanently true.

15. What's missing from the observability story today?
Real: CloudWatch logs, alarms, ALB/ECS/RDS metrics, and a purpose-built dashboard (`new-adv-rag-ops` , Section 28) — that's genuinely more than most portfolio RAG projects have. What's not there: distributed tracing. There's no request-level trace connecting "user asked X" → "router chose hybrid" → "RAG retrieval took Yms" → "SQL generation took Zms" → "final answer assembled," which means diagnosing why one specific slow request in the mixed-smoke test (Section 29.3, p95 34.05s) was slow requires reading logs and metadata rather than following one trace. Structured request-scoped logging with a correlation ID (which the `query_id` used for SQL

approvals could double as) plus something like OpenTelemetry would close that gap and directly help explain the latency finding that's already flagged as the project's main open issue.

16. Why `text-embedding-3-small` instead of `text-embedding-3-large`, and what did that trade off?
app/config.py sets `embedding_model: str = "text-embedding-3-small"`. The trade-off is the standard one for that model pair: `-small` is cheaper and faster per embedding call with a smaller vector dimensionality, `-large` gives measurably better retrieval quality on harder semantic-matching benchmarks at higher cost and storage size (larger vectors mean a larger Qdrant index). For a Kubernetes-docs-sized corpus where hybrid retrieval and reranking already compensate for weaker first-pass embeddings (coding Q1–4), `-small` is a reasonable cost/quality trade — but it's worth being able to say explicitly that it's a trade, not a free win, in case an interviewer asks "did you consider the large model?"

17. The follow-up rewriter resolves references like "the non-sensitive one" using conversation history — what's the injection/misuse risk there?
app/services/conversation_context.py rewrites ambiguous follow-ups by looking at the last turn and known subjects (ConfigMap, Secret, Pod, ServiceAccount, RBAC — coding Q12). Because this rewriting step feeds its output into intent routing and retrieval, a user could in principle try to manipulate multi-turn context to steer the rewritten question toward an unintended SQL or RAG path — for example, planting a misleading "subject" in an earlier turn so a later vague follow-up gets rewritten in a way that biases routing. This is a lower-severity risk than classic single-turn prompt injection because the rewritten question still goes through the same router, SQL validator (`sqlglot`), and approval gate as any other question — but it's a real surface that isn't explicitly tested today, and multi-turn manipulation is exactly the kind of thing input-safety tests in `tests/test_conversation_context.py` don't currently cover.

18. If you rebuilt this project from scratch, what would you do differently?
The strongest, most credible version of this answer is specific, not generic. Grounded in what's actually true about this codebase: I'd move the reranker blend weights and auto-retrieval thresholds out of code and into configuration from day one, rather than hardcoding them and only discovering later that they need to be tunable (Q14). I'd add connection pooling for Postgres from the start instead of raw `psycopg2.connect()` calls (Q3), since that's cheap to do early and annoying to retrofit. And I'd design the pending-SQL approval flow with row-level concurrency guards (Q12) from the first version, since human-in-the-loop approval is exactly the kind of feature where a race condition has real consequences, not just a UX glitch. Notice the pattern in all three: they're not "I'd use a different framework," they're "I'd apply the lesson this project already taught me, earlier." That's a much stronger answer than any hypothetical, because it's backed by the project's own history.

---

43. How This Was Actually Built: The Real Phase Order

Everything above explains what each piece does. This section explains the order they had to be built in, and why — pulled from `REBUILD_FROM_SCRATCH_GUIDE.md`. This matters for a different reason than the rest of the guide: "how does X work" and "how would you architect this from scratch" are different interview questions, and this section answers the second one.

43.1 The Rule

The guide states one rule bluntly:

> Do not start with `rag_service.py`.

Why? Because `rag_service.py` depends on retrieval services, which depend on ingestion, which depends on storage and cache primitives, which depend on shared config and models. Building the "interesting" part first means building it on top of nothing, then rewriting it repeatedly as the lower layers get filled in underneath it.

43.2 The 13-Phase Build Order

1. **Foundation** — `pyproject.toml`, `.env.example`, `Dockerfile`, `docker-compose.yml`, app bootstrap. Goal: FastAPI and Streamlit both boot locally against Docker-based Postgres and Qdrant.
2. **Configuration and shared contracts** — `Settings` in `app/config.py`, and shared models (`QueryRequest`, `ChatResponse`, `PendingSQLBlock`, `RetrievedChunk`, `CRAGEvaluation`, `ReflectionResult`) in `app/models.py`, plus graph state in `app/core/state.py`. Done early so nothing downstream is built around ad hoc dictionaries.

3. **Storage and cache primitives** — local/S3 storage backends, `QueryCacheService`, `DocCacheService`. This is also where the rule "query cache and document dedupe are separate services" gets established structurally, not just conceptually.
4. **Ingestion pipeline** — `document_processor.py`, `embedding_service.py`, `vector_store.py`, `ingestion_service.py`.
5. **Retrieval and search services** — `router_service.py`, `reranking.py`, `hyde.py`, `crag.py`, `self_reflective.py`, `web_search.py`. Dense/sparse/hybrid modes land in `vector_store.py` and `rag_service.py` first; reranking, HyDE, CRAG, and reflection are added on top once basic retrieval works.
6. **LLM, RAG, SQL, and auto retrieval** — `llm_service.py`, `rag_service.py`, `sql_service.py`, `auto_retrieval.py`, `system_prompt.py`.
7. **Security and middleware** — `auth.py`, `rate_limiter.py`, `input_guard.py`, `input_restructuring.py`, `content_moderation.py`, `output_validator.py`, `token_budget.py`, `spotlighting.py`.
8. **Orchestration graph** — `app/core/graph.py`, wiring `rag` / `sql` / `hybrid` routes together with pause-for-approval support.
9. **API routes** — `auth.py`, `query.py`, `admin.py`, `documents.py`. Rule: `/documents/upload` calls `IngestionService`; it does not reimplement ingestion inline.
10. **Scripts and local tooling** — seeders and admin scripts that call services rather than duplicating service logic.
11. **Tests and evaluation** — auth, cache, ingestion dedupe, SQL validation, vector store, RAG service, API, hybrid routing, admin, and eval-smoke coverage.
12. **Local end-to-end validation** — register, login, upload, RAG question, SQL question, hybrid question, approve SQL, repeat a question to confirm caching — all before touching AWS.
13. **Containerization, AWS deployment, monitoring, and load testing** — build and push images, stand up RDS/S3/Qdrant Cloud/Secrets Manager/ECS/ALB/Route 53/ACM, validate, add CloudWatch alarms and dashboard, then load test.

43.3 Why This Order Prevents Specific Failures

Each ordering choice maps to a failure mode that building out of order would cause:

- Building retrieval before storage/cache primitives exist means retrieval code gets rewritten once caching is retrofitted in.
- Building the RAG path before ingestion is centralized means ingestion logic tends to leak into routes and scripts, which is exactly the anti-pattern Section 9 and coding Q35 warn against.
- Building AWS deployment before local end-to-end validation means cloud-only bugs get tangled up with application-logic bugs, making both harder to isolate.

43.4 Common Mistakes to Avoid

This list is worth memorizing as-is, since it reads as a checklist an interviewer would respect:

- starting with high-level RAG logic before lower-level services exist
- mixing document dedupe with query cache logic
- scattering ingestion logic across routes and scripts
- using only regex SQL guards instead of AST-based validation
- assuming healthy containers imply healthy ALB targets
- forgetting RDS security-group ingress from the ECS service
- leaving Streamlit pointed at localhost after deployment
- assuming CloudWatch auto-created scaling alarms are the same as manually created alarms
- treating one successful local run as sufficient validation

---

44. Three More Production Incidents Worth Knowing

Section 31 already covers the CI/Postgres test dependency incident. Three more real incidents are documented in `WRITEUP.md` and are just as strong for interviews — each one is a specific bug with a specific root cause and a specific fix, not a generic "things broke sometimes" story.

44.1 The UTF-8 Encoding Bug That Caused Intermittent Timeouts

What happened: After deployment, requests on heavier code paths intermittently failed with timeouts, even though the backend's `/health` endpoint stayed green.

Root cause: stdout/stderr encoding behavior in the container runtime. The container environment did not consistently assume UTF-8, and under certain conditions this affected request handling on the heavier paths enough to cause timeouts.

Fix: force UTF-8 explicitly rather than relying on the container's default locale — setting `PYTHONIOENCODING=utf-8`, `PYTHONUTF8=1`, `LANG=C.UTF-8`, and `LC_ALL=C.UTF-8` in the backend task definition's environment, and reconfiguring stdout/stderr encoding safely during backend startup.

Why this is a good interview story: it is the kind of bug that looks like a networking or timeout problem at first (and would send someone straight to checking ALB health checks and security groups) but was actually a runtime/locale issue several layers away from where the symptom showed up. It demonstrates debugging discipline — not stopping at "it times out sometimes" but tracing the failure to its actual cause.

44.2 Healthy Containers, Unhealthy ALB Targets

What happened: Containers could be running and reporting healthy at the process level while still failing ALB target-group health checks, blocking traffic from ever reaching them.

Root cause: subnet, security-group, and target-group alignment. ECS service subnets need to align with the ALB-enabled availability zones, and security-group ingress needs to explicitly allow ALB-to-task traffic on the container port. A container can be perfectly healthy and still be unreachable if either of those is misconfigured.

Fix: corrected security-group ingress rules, aligned ECS service subnets with the ALB's availability zones, and validated target-group health directly rather than inferring it from container status alone.

Why this matters: it is the concrete version of the "common mistake" listed in Section 43.4 — "assuming healthy containers imply healthy ALB targets." Container health and load-balancer-visible health are two different signals, and conflating them is a very common early-AWS mistake worth being able to name specifically.

44.3 Streamlit Serving Stale Code After Deployment

What happened: after a deployment, the live Streamlit frontend appeared to still be running older code or configuration than what was in the repository.

Root cause: a combination of Docker build-context issues (building from the wrong parent workspace could pull in the wrong Streamlit file) and a stale `API_BASE_URL` environment value left pointing at the wrong backend.

Fix: corrected the build-context assumptions so images are always built from the `new-adv-rag-implement` directory, corrected the task-definition environment values, and revalidated the live frontend directly against the HTTPS backend domain after redeploying — rather than trusting that a new deployment automatically means new code is actually running.

Why this matters: "the deploy succeeded" and "the deployed thing is running the code you think it's running" are not the same claim, and this incident is a concrete example of verifying the second one instead of assuming it from the first.